

Project Progress Report: Week 7

Project Title: CodeDNA – Biomorphic Code Visualization Engine

Task Domain: Basic Visualization — Initial SVG Rendering

1. Basic Visualization (Week 7)

Week 7 introduces the `SVGBuilder` class, the first rendering stage of the CodeDNA pipeline. Where Week 6's `CreatureGenerator` only decided what an organism should look like (its data model), `SVGBuilder` is responsible for actually drawing it — translating a fully-specified `Creature` object into a real, static SVG image. This week's scope is deliberately limited to structure: producing a correctly composed SVG document and a viewable HTML wrapper, with no motion or animation yet. Animated behaviour is scoped for Week 8.

1.1 SVG Structure Generation

The `build()` method assembles the final SVG document by calling a sequence of private helper methods, each responsible for one visual layer of the organism. The layers are drawn in back-to-front order so that the body and nucleus correctly sit on top of the aura ring, and mutations render above the body:

```
svg.append(buildAura(c)); // outer health-ring, drawn first (background layer)

svg.append(buildCells(c)); // loop / if dots
svg.append(buildBody(c)); // main organism blob
svg.append(buildMutations(c)); // nested-loop warning dots
svg.append(buildNucleus(c)); // class-core circle, drawn last (top layer)
svg.append(buildLabel(c)); // organism name
```

The canvas uses a fixed 800×700 `viewBox` with the organism centred at (400, 350), matching the coordinate system that later weeks' animated version will reuse without modification — an important design choice, since it means Week 8 can add motion to these same elements without recalculating any positions.

1.2 Organism Shape Mapping

Each SVG element is drawn directly from fields already computed by Week 6's Creature object, so no new metrics or calculations are introduced this week — only rendering logic:

SVG Element	Source & Meaning
Aura ring	A stroked circle of radius <code>auraRadius</code> , coloured by <code>auraColor</code> , with opacity <code>auraOpacity</code> — visualises the health score as a halo.
Body	A filled circle of radius <code>bodyRadius</code> using <code>bodyColor</code> and <code>bodyGlow</code> as its stroke — the organism's main mass, sized from code line count.
Nucleus	A smaller filled circle of radius <code>nucleusRadius</code> in <code>nucleusColor</code> , centred inside the body — represents the class core.
Cells	One small circle per loop/if statement, positioned at each cell's fixed (cx, cy) offset from centre and coloured cyan (loops) or purple (ifs).
Mutations	One small red circle per nested loop, offset from centre — flags structurally risky code.
Label	A text element beneath the body showing the organism's name (derived from the source file name).

1.3 Static Rendering Approach

Unlike the final version of SVGBuilder, this week's implementation intentionally omits `<animate>` and `<animateMotion>` elements, gradient definitions, and glow filters. Every shape is drawn with fixed `cx/cy/r` attributes computed once at build time. This keeps the first rendering milestone focused on a single question — does the creature data model correctly translate into a coherent visual shape? — before layering in motion, gradients, and orbiting paths, which depend on this static structure being correct first. Tentacles (method mapping to limbs) are also deferred to Week 8, since they require the more complex curved-path math that animation will drive.

1.4 HTML Export for Browser Viewing

A minimal HTMLExporter wraps the generated SVG in a bare HTML page — dark background, centred layout — purely so the organism can be opened and viewed in a browser. No dashboard, statistics panel, or styling beyond basic centring is included at this stage; the full HUD-style report layout is planned for Week 9, once parsing, mapping, and visualization are integrated together.

2. OOP Concepts Applied

- Single Responsibility — SVGBuilder only renders; it performs no parsing or data-mapping logic, both of which remain in ASTParser and CreatureGenerator respectively.
- Encapsulation — each visual layer (aura, cells, body, mutations, nucleus, label) is built by its own private helper method, keeping build() as a short, readable composition of parts.
- Abstraction — HTMLExporter.export() takes a Creature and SVG string and returns a file path, hiding all HTML/CSS construction details from the caller (Main).
- Separation of Concerns — this week cleanly completes the pipeline boundary: ASTParser parses, CreatureGenerator maps, SVGBuilder renders, HTMLExporter packages for viewing.
- Static Methods — build() and export() remain static utility entry points, consistent with the design established in Weeks 3–6.

3. Development Summary

By the end of Week 7, the CodeDNA project has its first visible output: a real, static SVG organism generated directly from a Java source file's structure, viewable in any browser via the accompanying HTML export. This closes the loop from raw code to rendered image for the first time, validating that the full pipeline — ASTParser → CreatureGenerator → SVGBuilder → HTMLExporter — is wired correctly end to end. The organism currently has no motion, gradients, or tentacles; these visual enhancements, along with the dashboard-style HTML report, are scoped for Weeks 8 and 9. The project remains on track, with the foundational rendering structure now in place to build the animated version on top of.